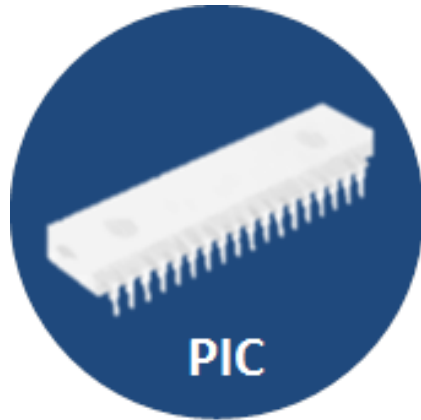


Tema 2. Arquitectura del PIC



Contenido:

Características de PIC18F4550

Organización de la memoria de datos

Registros de funciones especiales (SFR)

Registros de propósito general (GPR)

Registro TRIS_

Registro PORT_

Registro LAT_

Registro ADCON1

Direccionamiento

Memoria de programa

Oscilador

Reset

Palabra de configuración

Sistema mínimo

Diagrama de bloque



Arquitectura del PIC18F4550

Características de PIC18F4550

Device	Program Memory		Data Memory		I/O	10-Bit A/D (ch)	CCP/ECCP (PWM)	SPP	MSSP		EUSART	Comparators	Timers 8/16-Bit
	Flash (bytes)	# Single-Word Instructions	SRAM (bytes)	EEPROM (bytes)					SPI	Master I ² C™			
PIC18F2455	24K	12288	2048	256	24	10	2/0	No	Y	Y	1	2	1/3
PIC18F2550	32K	16384	2048	256	24	10	2/0	No	Y	Y	1	2	1/3
PIC18F4455	24K	12288	2048	256	35	13	1/1	Yes	Y	Y	1	2	1/3
PIC18F4550	32K	16384	2048	256	35	13	1/1	Yes	Y	Y	1	2	1/3

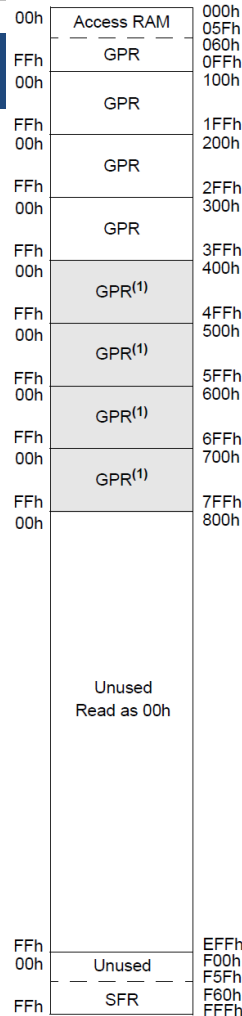


Arquitectura del PIC18F4550

Memoria de datos

[illegible]

Memoria de datos



The BSR specifies the bank used by the instruction.

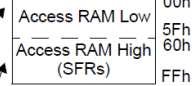


TABLE 5-1: SPECIAL FUNCTION REGISTER MAP

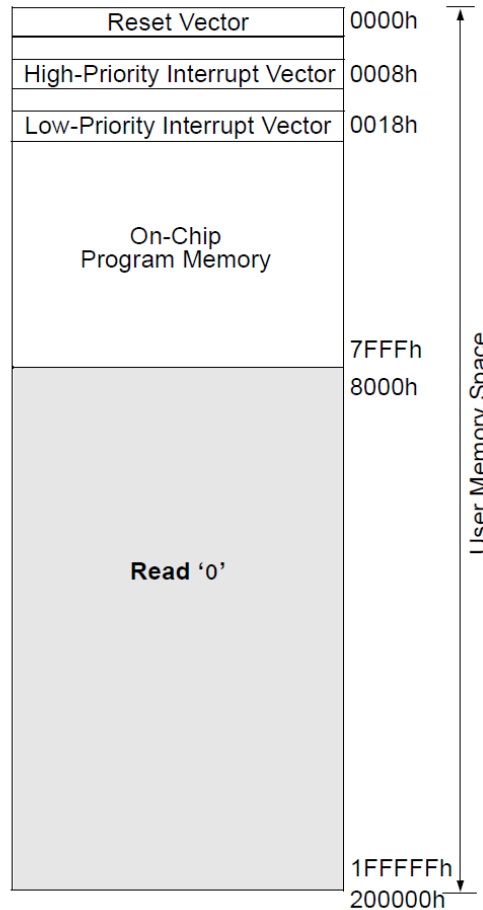
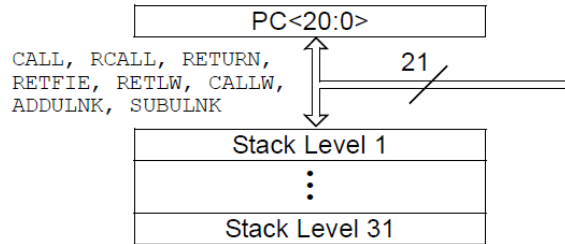
Address	Name	Address	Name	Address	Name	Address	Name	Address	Name
FFFh	TOSU	FDFh	INDF2 ⁽¹⁾	FBFh	CCPR1H	F9Fh	IPR1	F7Fh	UEP15
FFEh	TOSH	FDEh	POSTINC2 ⁽¹⁾	FBEh	CCPR1L	F9Eh	PIR1	F7Eh	UEP14
FFDh	TOSL	FDDh	POSTDEC2 ⁽¹⁾	FBDh	CCP1CON	F9Dh	PIE1	F7Dh	UEP13
FFCh	STKPTR	FDCCh	PREINC2 ⁽¹⁾	FBCCh	CCPR2H	F9Ch	— ⁽²⁾	F7Ch	UEP12
FFBh	PCLATU	FDBh	PLUSW2 ⁽¹⁾	FBBh	CCPR2L	F9Bh	OSCTUNE	F7Bh	UEP11
FFAh	PCLATH	FDAh	FSR2H	FBAh	CCP2CON	F9Ah	— ⁽²⁾	F7Ah	UEP10
FF9h	PCL	FD9h	FSR2L	FB9h	— ⁽²⁾	F99h	— ⁽²⁾	F79h	UEP9
FF8h	TBLPTRU	FD8h	STATUS	FB8h	BAUDCON	F98h	— ⁽²⁾	F78h	UEP8
FF7h	TBLPTRH	FD7h	TMR0H	FB7h	ECCP1DEL	F97h	— ⁽²⁾	F77h	UEP7
FF6h	TBLPTRL	FD6h	TMR0L	FB6h	ECCP1AS	F96h	TRISE ⁽³⁾	F76h	UEP6
FF5h	TABLAT	FD5h	T0CON	FB5h	CVRCON	F95h	TRISD ⁽³⁾	F75h	UEP5
FF4h	PRODH	FD4h	— ⁽²⁾	FB4h	CMCON	F94h	TRISC	F74h	UEP4
FF3h	PRODL	FD3h	OSCCON	FB3h	TMR3H	F93h	TRISB	F73h	UEP3
FF2h	INTCON	FD2h	HLVDCON	FB2h	TMR3L	F92h	TRISA	F72h	UEP2
FF1h	INTCON2	FD1h	WDTCON	FB1h	T3CON	F91h	— ⁽²⁾	F71h	UEP1
FF0h	INTCON3	FD0h	RCON	FB0h	SPBRGH	F90h	— ⁽²⁾	F70h	UEP0
FEFh	INDF0 ⁽¹⁾	FCFh	TMR1H	FAFh	SPBRG	F8Fh	— ⁽²⁾	F6Fh	UCFG
FEeh	POSTINC0 ⁽¹⁾	FCEh	TMR1L	FAeh	RCREG	F8Eh	— ⁽²⁾	F6Eh	UADDR
FEDh	POSTDEC0 ⁽¹⁾	FCDh	T1CON	FADh	TXREG	F8Dh	LATE ⁽³⁾	F6Dh	UCON
FECh	PREINC0 ⁽¹⁾	FCCh	TMR2	FACH	TXSTA	F8Ch	LATD ⁽³⁾	F6Ch	USTAT
FEbh	PLUSW0 ⁽¹⁾	FCBh	PR2	FABh	RCSTA	F8Bh	LATC	F6Bh	UEIE
FEAh	FSROH	FCAh	T2CON	FAAh	— ⁽²⁾	F8Ah	LATB	F6Ah	UEIR
FE9h	FSROL	FC9h	SSPBUF	FA9h	EEADR	F89h	LATA	F69h	UIE
FE8h	WREG	FC8h	SSPAD	FA8h	EEDATA	F88h	— ⁽²⁾	F68h	UIR
FE7h	INDF1 ⁽¹⁾	FC7h	SSPSTAT	FA7h	EECON2 ⁽¹⁾	F87h	— ⁽²⁾	F67h	UFRMH
FE6h	POSTINC1 ⁽¹⁾	FC6h	SSPCON1	FA6h	EECON1	F86h	— ⁽²⁾	F66h	UFRML
FE5h	POSTDEC1 ⁽¹⁾	FC5h	SSPCON2	FA5h	— ⁽²⁾	F85h	— ⁽²⁾	F65h	SPPCON ⁽³⁾
FE4h	PREINC1 ⁽¹⁾	FC4h	ADRESH	FA4h	— ⁽²⁾	F84h	PORTE	F64h	SPPEPS ⁽³⁾
FE3h	PLUSW1 ⁽¹⁾	FC3h	ADRESL	FA3h	— ⁽²⁾	F83h	PORTD ⁽³⁾	F63h	SPPCFG ⁽³⁾
FE2h	FSR1H	FC2h	ADCON0	FA2h	IPR2	F82h	PORTC	F62h	SPPDATA ⁽³⁾
FE1h	FSR1L	FC1h	ADCON1	FA1h	PIR2	F81h	PORTB	F61h	— ⁽²⁾
FE0h	BSR	FC0h	ADCON2	FA0h	PIE2	F80h	PORTA	F60h	— ⁽²⁾

Note 1: Not a physical register.
2: Unimplemented registers are read as '0'.
3: These registers are implemented only on 40/44-pin devices.



Arquitectura del PIC18F4550

Memoria de programa





Arquitectura del PIC18F4550

Memoria de programa

FIGURE 5-4: INSTRUCTIONS IN PROGRAM MEMORY

			Word Address		
			LSB = 1	LSB = 0	↓
					000000h
					000002h
					000004h
					000006h
Instruction 1:	MOVLW	055h	0Fh	55h	000008h
Instruction 2:	GOTO	0006h	EFh	03h	00000Ah
			F0h	00h	00000Ch
Instruction 3:	MOVFF	123h, 456h	C1h	23h	00000Eh
			F4h	56h	000010h
					000012h
					000014h

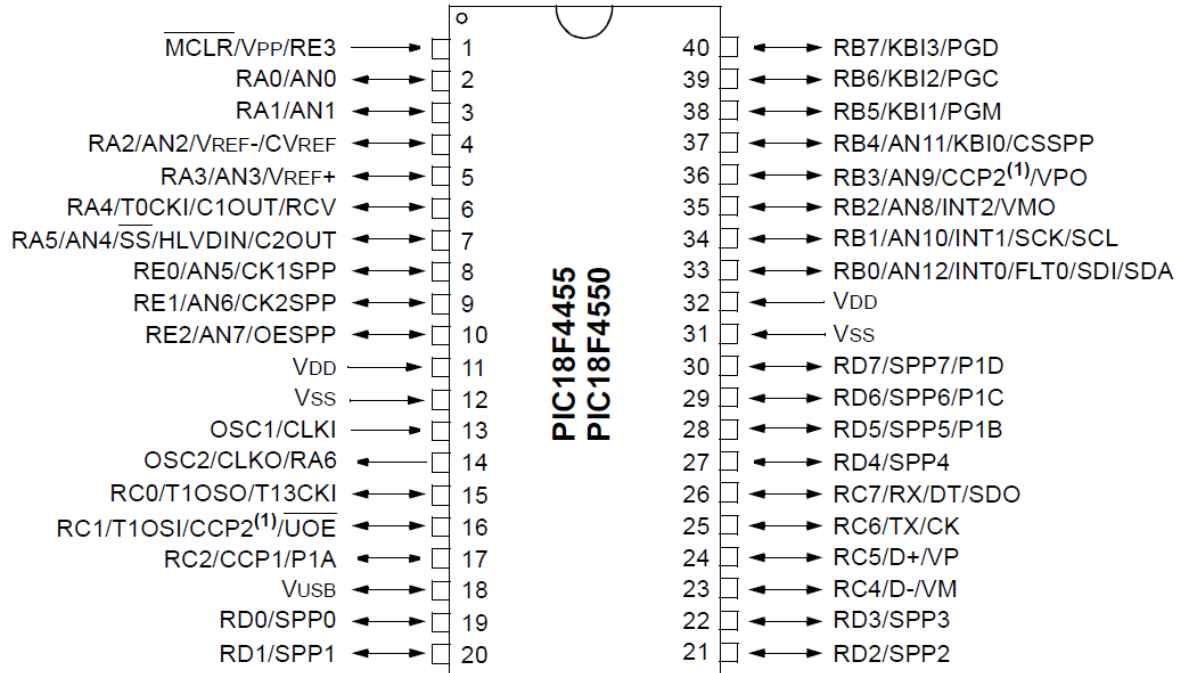
EXAMPLE 5-4: TWO-WORD INSTRUCTIONS

CASE 1:		
Object Code	Source Code	
0110 0110 0000 0000	TSTFSZ	REG1 ; is RAM location 0?
1100 0001 0010 0011	MOVFF	REG1, REG2 ; No, skip this word
1111 0100 0101 0110		; Execute this word as a NOP
0010 0100 0000 0000	ADDWF	REG3 ; continue code
CASE 2:		
Object Code	Source Code	
0110 0110 0000 0000	TSTFSZ	REG1 ; is RAM location 0?
1100 0001 0010 0011	MOVFF	REG1, REG2 ; Yes, execute this word
1111 0100 0101 0110		; 2nd word of instruction
0010 0100 0000 0000	ADDWF	REG3 ; continue code



Arquitectura del PIC18F4550

Diagrama





Arquitectura del PIC18F4550

Registro ADCON1

REGISTER 21-2: ADCON1: A/D CONTROL REGISTER 1

U-0	U-0	R/W-0	R/W-0	R/W-0 ⁽¹⁾	R/W ⁽¹⁾	R/W ⁽¹⁾	R/W ⁽¹⁾
—	—	VCFG1	VCFG0	PCFG3	PCFG2	PCFG1	PCFG0
bit 7							bit 0

PCFG3: PCFG0	AN12	AN11	AN10	AN9	AN8	AN7 ⁽²⁾	AN6 ⁽²⁾	AN5 ⁽²⁾	AN4	AN3	AN2	AN1	AN0
0000 ⁽¹⁾	A	A	A	A	A	A	A	A	A	A	A	A	A
0001	A	A	A	A	A	A	A	A	A	A	A	A	A
0010	A	A	A	A	A	A	A	A	A	A	A	A	A
0011	D	A	A	A	A	A	A	A	A	A	A	A	A
0100	D	D	A	A	A	A	A	A	A	A	A	A	A
0101	D	D	D	A	A	A	A	A	A	A	A	A	A
0110	D	D	D	D	A	A	A	A	A	A	A	A	A
0111 ⁽¹⁾	D	D	D	D	D	A	A	A	A	A	A	A	A
1000	D	D	D	D	D	D	A	A	A	A	A	A	A
1001	D	D	D	D	D	D	D	A	A	A	A	A	A
1010	D	D	D	D	D	D	D	D	A	A	A	A	A
1011	D	D	D	D	D	D	D	D	D	A	A	A	A
1100	D	D	D	D	D	D	D	D	D	D	A	A	A
1101	D	D	D	D	D	D	D	D	D	D	D	A	A
1110	D	D	D	D	D	D	D	D	D	D	D	D	A
1111	D	D	D	D	D	D	D	D	D	D	D	D	D

A = Analog input

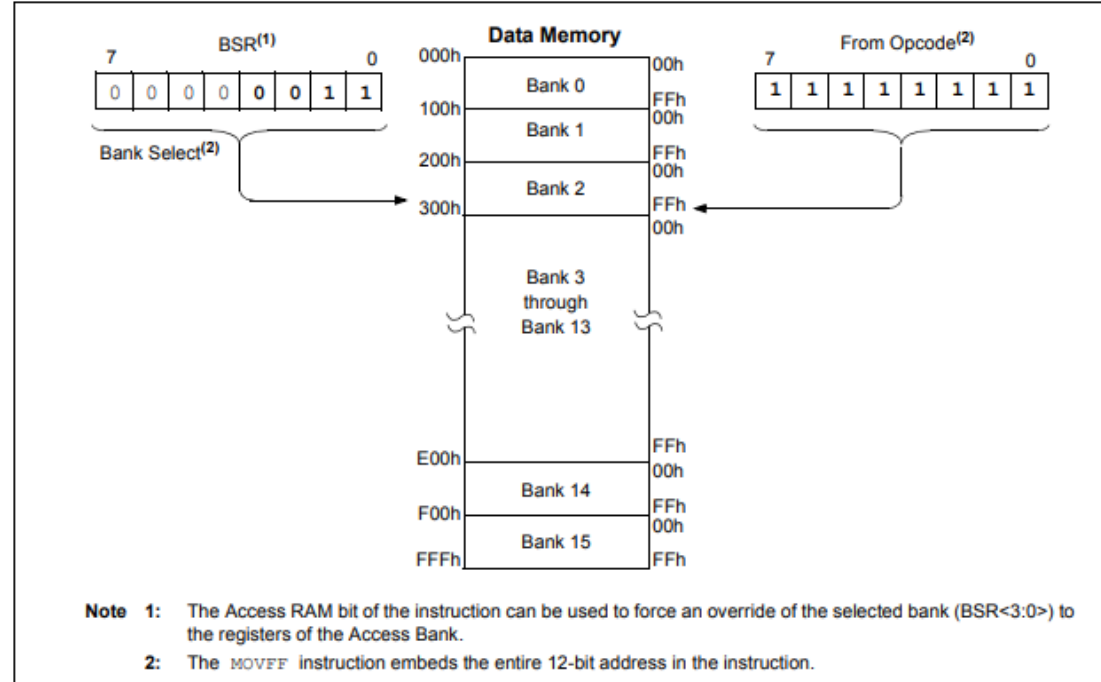
D = Digital I/O



Arquitectura del PIC18F4550

Direcccionamiento

FIGURE 5-6: USE OF THE BANK SELECT REGISTER (DIRECT ADDRESSING)





Arquitectura del PIC18F4550

Direccionamiento

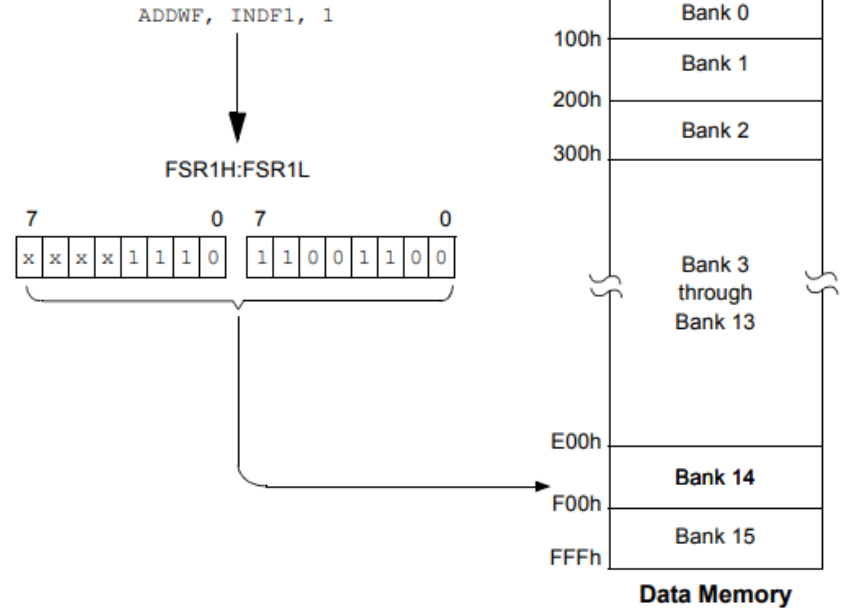
FIGURE 5-7: INDIRECT ADDRESSING

Using an instruction with one of the indirect addressing registers as the operand....

...uses the 12-bit address stored in the FSR pair associated with that register....

...to determine the data memory location to be used in that operation.

In this case, the FSR1 pair contains ECCh. This means the contents of location ECCh will be added to that of the W register and stored back in ECCh.

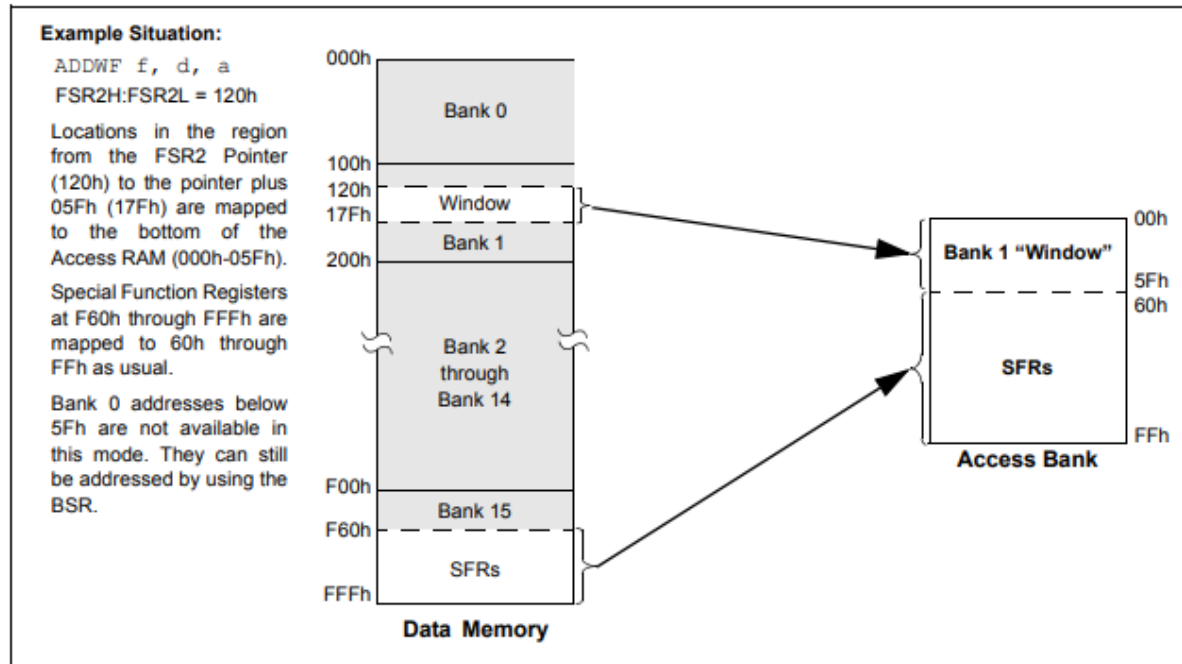




Arquitectura del PIC18F4550

Direccionamiento

FIGURE 5-9: REMAPPING THE ACCESS BANK WITH INDEXED LITERAL OFFSET ADDRESSING





Arquitectura del PIC18F4550

Oscilador

2.2 Oscillator Types

PIC18F2455/2550/4455/4550 devices can be operated in twelve distinct oscillator modes. In contrast with previous PIC18 enhanced microcontrollers, four of these modes involve the use of two oscillator types at once. Users can program the FOSC3:FOSC0 Configuration bits to select one of these modes:

1. XT Crystal/Resonator
2. HS High-Speed Crystal/Resonator
3. HSPLL High-Speed Crystal/Resonator with PLL Enabled
4. EC External Clock with Fosc/4 Output
5. ECIO External Clock with I/O on RA6
6. ECPLL External Clock with PLL Enabled and Fosc/4 Output on RA6
7. ECPIO External Clock with PLL Enabled, I/O on RA6
8. INTHS Internal Oscillator used as Microcontroller Clock Source, HS Oscillator used as USB Clock Source
9. INTIO Internal Oscillator used as Microcontroller Clock Source, EC Oscillator used as USB Clock Source, Digital I/O on RA6
10. INTCKO Internal Oscillator used as Microcontroller Clock Source, EC Oscillator used as USB Clock Source, Fosc/4 Output on RA6

TABLE 2-2: CAPACITOR SELECTION FOR CRYSTAL OSCILLATOR

Osc Type	Crystal Freq	Typical Capacitor Values Tested:	
		C1	C2
XT	4 MHz	27 pF	27 pF
HS	4 MHz	27 pF	27 pF
	8 MHz	22 pF	22 pF
	20 MHz	15 pF	15 pF



Arquitectura del PIC18F4

Oscilador

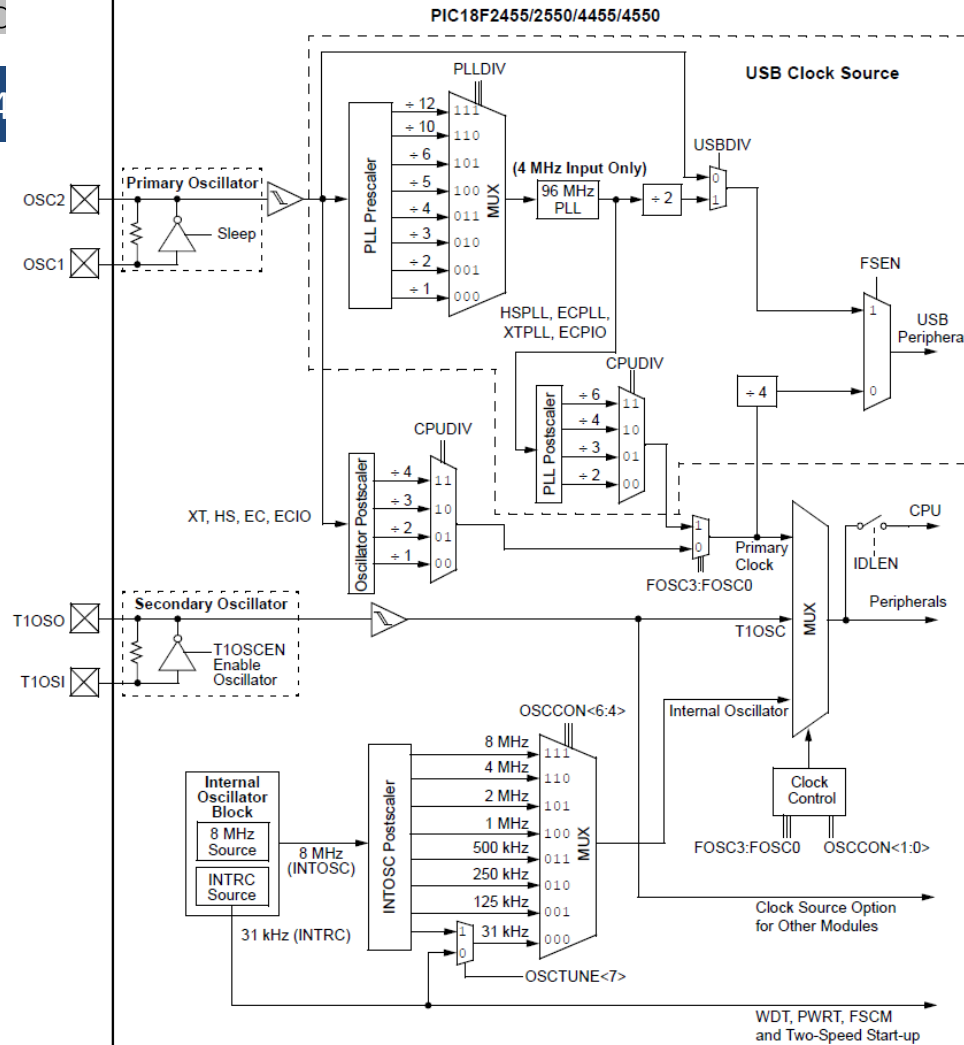
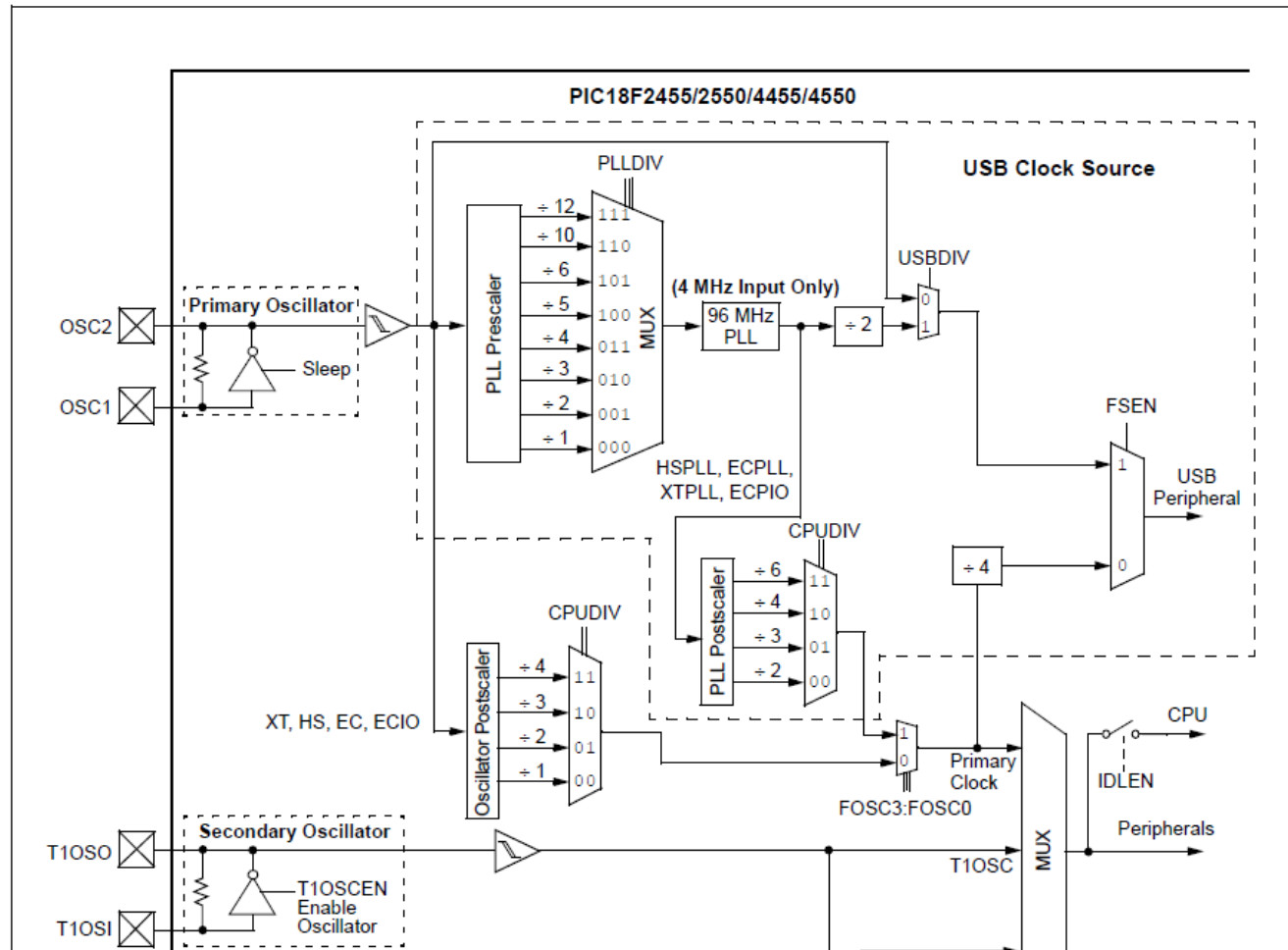


FIGURE 2-1:

PIC18F2455/2550/4455/4550 CLOCK DIAGRAM



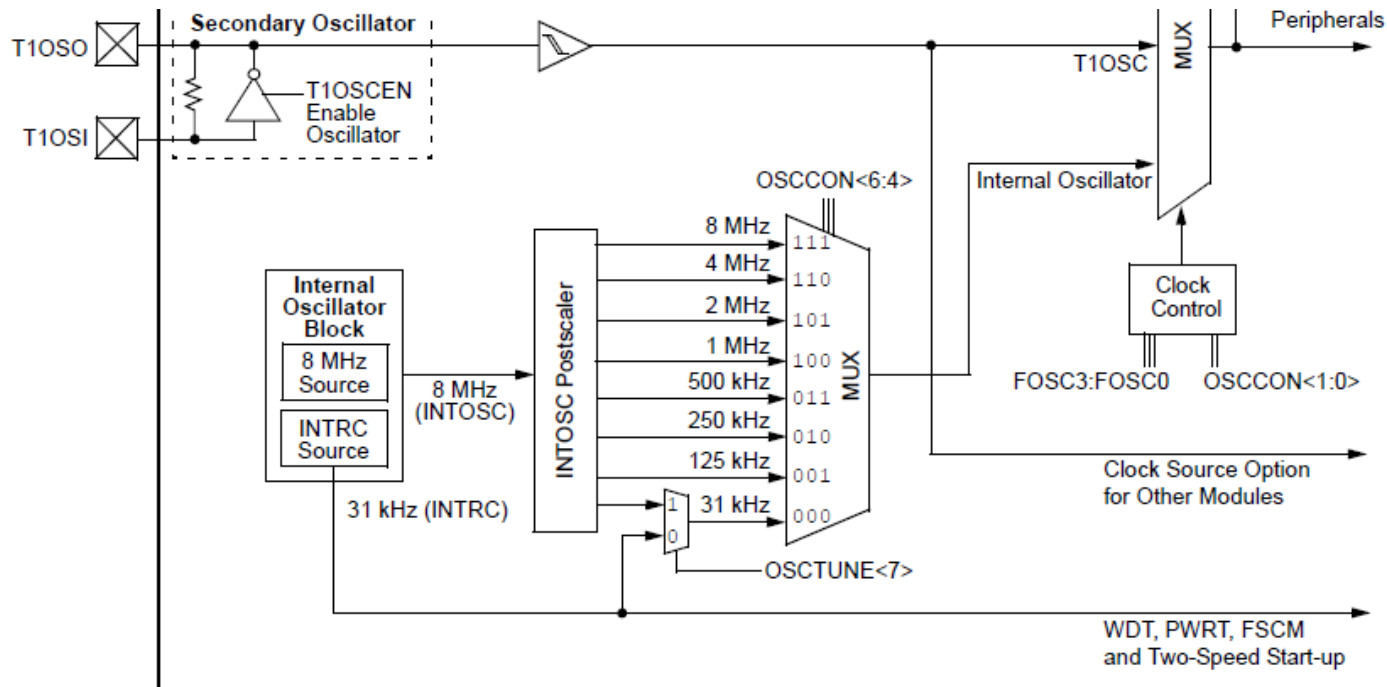
Oscilador





Arquitectura del PIC18F4550

Oscilador





Arquitectura del PIC18F4550

Reset

Tipos de Reset en el PIC18F4550

1. Reset por Hardware (MCLR)
2. Reset por Software
3. Reset por Watchdog Timer (WDT)
4. Reset por Fallo de Energía (BOR - Brown-Out Reset)
5. Reset por Encendido (POR - Power-On Reset)
6. Reset por temporizador (PWRT - POWER-UP TIMER)

4.0 RESET

The PIC18F2455/2550/4455/4550 devices differentiate between various kinds of Reset:

- a) Power-on Reset (POR)
- b) MCLR Reset during normal operation
- c) MCLR Reset during power-managed modes
- d) Watchdog Timer (WDT) Reset (during execution)
- e) Programmable Brown-out Reset (BOR)
- f) `RESET` Instruction
- g) Stack Full Reset
- h) Stack Underflow Reset

This section discusses Resets generated by MCLR, POR and BOR and covers the operation of the various start-up timers. Stack Reset events are covered in **Section 5.1.2.4 “Stack Full and Underflow Resets”**. WDT Resets are covered in **Section 25.2 “Watchdog Timer (WDT)”**.



Arquitectura del PIC18F4550

Palabra de Configuración

```

CONFIG FOSC=HS
CONFIG PWRT=ON
CONFIG BOR=OFF
CONFIG WDT=OFF
CONFIG MCLRE=ON
CONFIG PBADEN=OFF
CONFIG DEBUG=OFF
CONFIG XINST=OFF
  
```

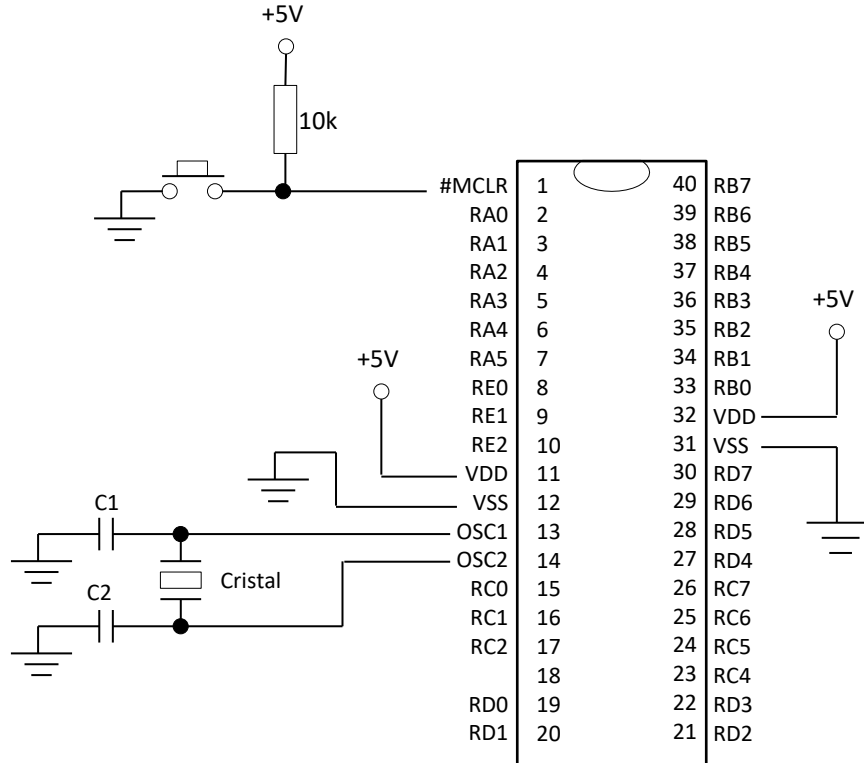
TABLE 25-1: CONFIGURATION BITS AND DEVICE IDs

File Name		Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Default/ Unprogrammed Value
300000h	CONFIG1L	—	—	USBDIV	CPUDIV1	CPUDIV0	PLLDIV2	PLLDIV1	PLLDIV0	--00 0000
300001h	CONFIG1H	IESO	FCMEN	—	—	FOSC3	FOSC2	FOSC1	FOSC0	00-- 0101
300002h	CONFIG2L	—	—	VREGEN	BORV1	BORV0	BOREN1	BOREN0	PWRTEN	--01 1111
300003h	CONFIG2H	—	—	—	WDTPS3	WDTPS2	WDTPS1	WDTPS0	WDTEN	---1 1111
300005h	CONFIG3H	MCLRE	—	—	—	—	LPT1OSC	PBADEN	CCP2MX	1--- -011
300006h	CONFIG4L	DEBUG	XINST	ICPRT ⁽³⁾	—	—	LVP	—	STVREN	100- -1-1
300008h	CONFIG5L	—	—	—	—	CP3 ⁽¹⁾	CP2	CP1	CP0	---- 1111
300009h	CONFIG5H	CPD	CPB	—	—	—	—	—	—	11-- ----
30000Ah	CONFIG6L	—	—	—	—	WRT3 ⁽¹⁾	WRT2	WRT1	WRT0	---- 1111
30000Bh	CONFIG6H	WRD	WRTB	WRTC	—	—	—	—	—	111- ----
30000Ch	CONFIG7L	—	—	—	—	EBTR3 ⁽¹⁾	EBTR2	EBTR1	EBTR0	---- 1111
30000Dh	CONFIG7H	—	EBTRB	—	—	—	—	—	—	-1-- ----
3FFFFEh	DEVID1	DEV2	DEV1	DEV0	REV4	REV3	REV2	REV1	REV0	xxxx xxxx ⁽²⁾
3FFFFFh	DEVID2	DEV10	DEV9	DEV8	DEV7	DEV6	DEV5	DEV4	DEV3	0001 0010 ⁽²⁾



Arquitectura del PIC18F4550

Sistema mínimo





Arquitectura del PIC18F4

Diagrama de bloque

FIGURE 1-2: PIC18F4455/4550 (40/44-PIN) BLOCK DIAGRAM

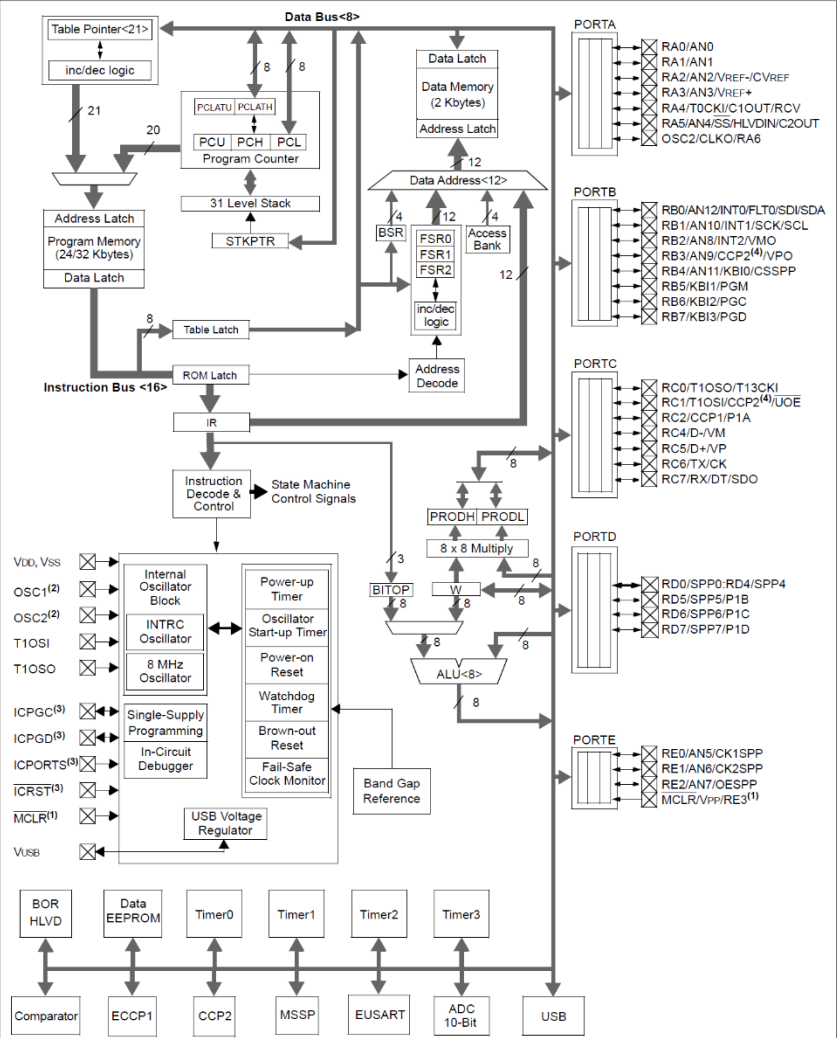


FIGURE 1-2: PIC18F4455/4550 (40/44-PIN) BLOCK DIAGRAM



Arquitectura del PIC18F4

Diagrama de bloque

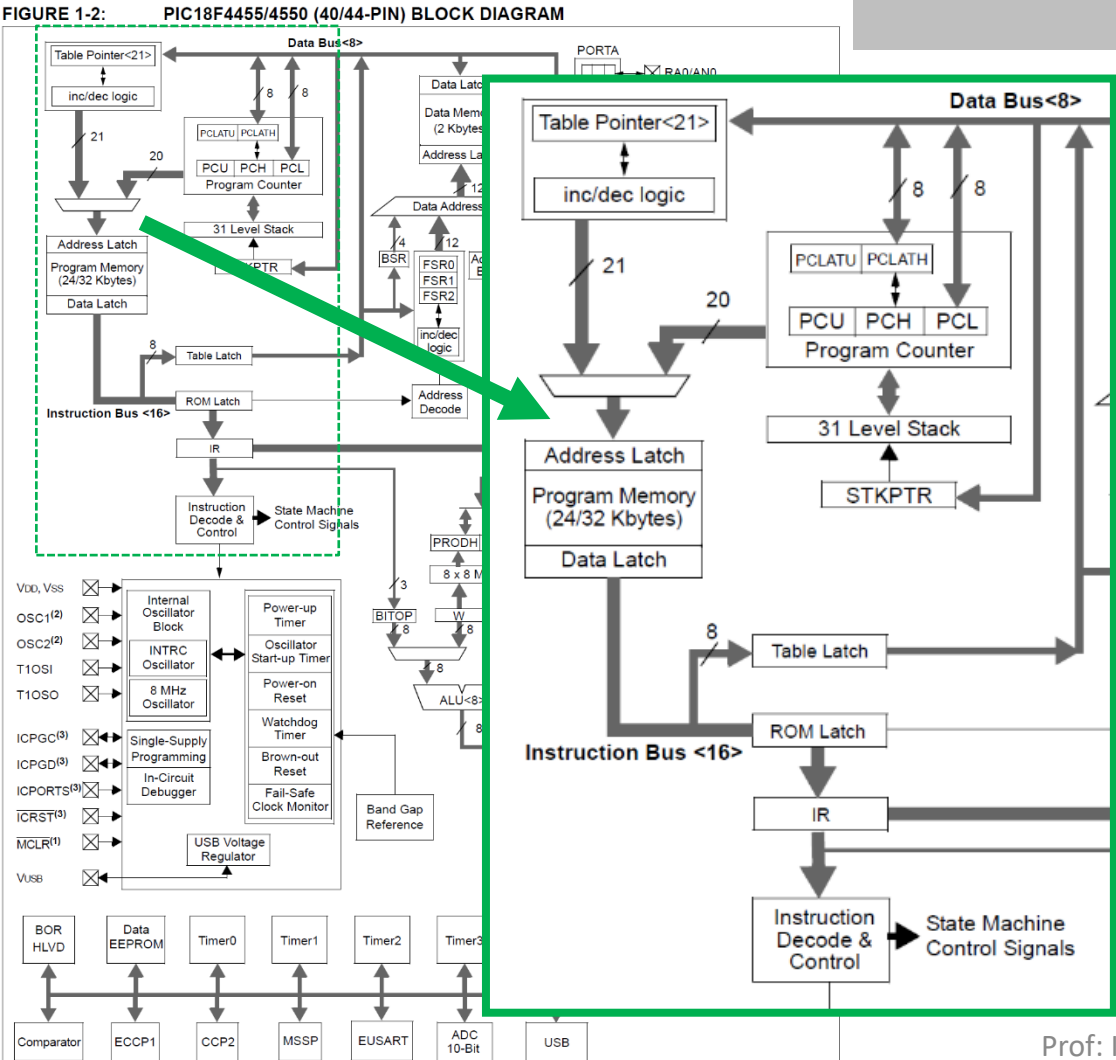


Diagrama de bloque

The diagram illustrates the internal architecture of the PIC16C55 microcontroller, centered around a **Data Bus <8>** and an **Instruction Bus <16>**.

Internal Components:

- Table Pointer <21>**: Connected to **inc/dec logic** and the **Data Bus** via an 8-bit bus.
- Program Counter**: Consists of **PCLATH**, **PCLATU**, **PCU**, **PCH**, and **PCL**. It is connected to the **Data Bus** via an 8-bit bus and to the **31 Level Stack**.
- 31 Level Stack**: Connected to the **Program Counter** and the **STKPTR** register.
- STKPTR**: Stack Pointer register.
- Address Latch**: Receives a 21-bit address from the **Table Pointer** and outputs it to the **Program Memory (24/32 Kbytes)**.
- Program Memory (24/32 Kbytes)**: Connected to the **Address Latch** and the **Data Bus** via an 8-bit bus.
- Data Latch**: Connected to the **Program Memory** and the **Data Bus** via an 8-bit bus.
- Table Latch**: Connected to the **Data Bus** via an 8-bit bus.
- Data Memory (2 Kbytes)**: Connected to the **Data Bus** via an 8-bit bus.
- Address Latch**: Receives a 12-bit address from the **Data Memory** and outputs it to the **Data Address <12>**.
- Data Address <12>**: Connected to the **Data Bus** via a 12-bit bus.
- Access Bank**: Connected to the **Data Address** and the **Data Bus** via a 4-bit bus.
- FSR0, FSR1, FSR2**: File Select Registers connected to the **Access Bank** and the **Data Bus** via a 4-bit bus.
- inc/dec logic**: Connected to the **FSRs** and the **Data Bus** via an 8-bit bus.
- Address**: Output from the **inc/dec logic** to the **Program Memory**.

External Interfaces:

- PORTA**: 8-bit port with pins RA0/AN0, RA1/AN1, RA2/AN2/VREF-/CVREF, RA3/AN3/VREF+, RA4/TOCKI/C1OUT/RCV, RA5/AN4/SSHLVDIN/C2OUT, and OSC2/CLKO/RA6.
- PORTB**: 8-bit port with pins RB0/AN12/INT0/FLT0/SDI/SDO, RB1/AN10/INT1/SCK/SCL, RB2/AN8/INT2/VMO, RB3/AN9/CCP2⁽⁴⁾/P0, RB4/AN11/KBI0/CSSPP, RB5/KBI1/PGM, RB6/KBI2/PGC, and RB7/KBI3/PGD.
- PORTC**: 8-bit port.

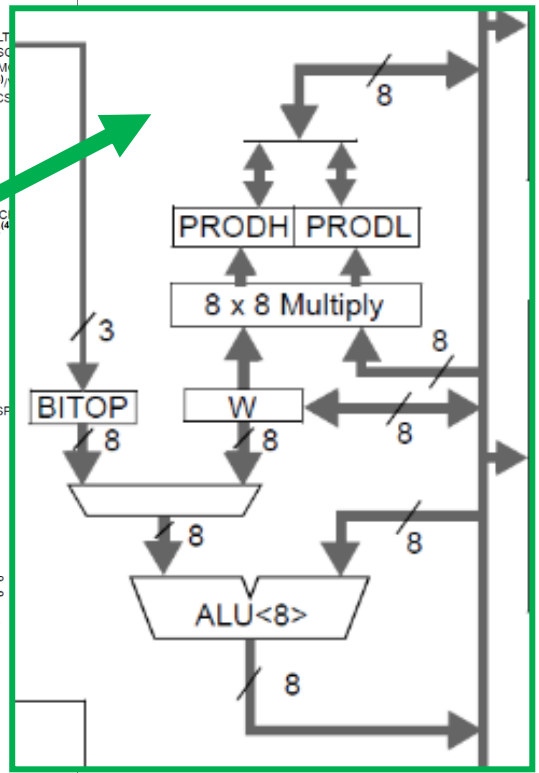
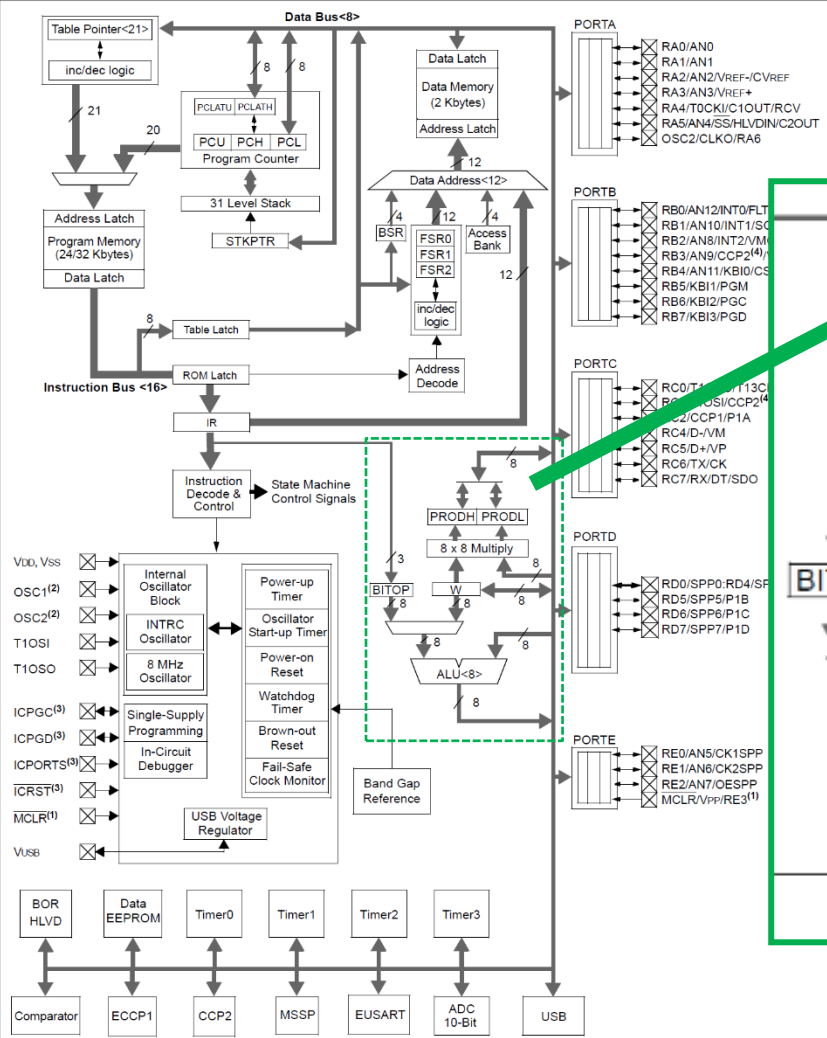




Arquitectura del PIC18F4

Diagrama de bloque

FIGURE 1-2: PIC18F4455/4550 (40/44-PIN) BLOCK DIAGRAM

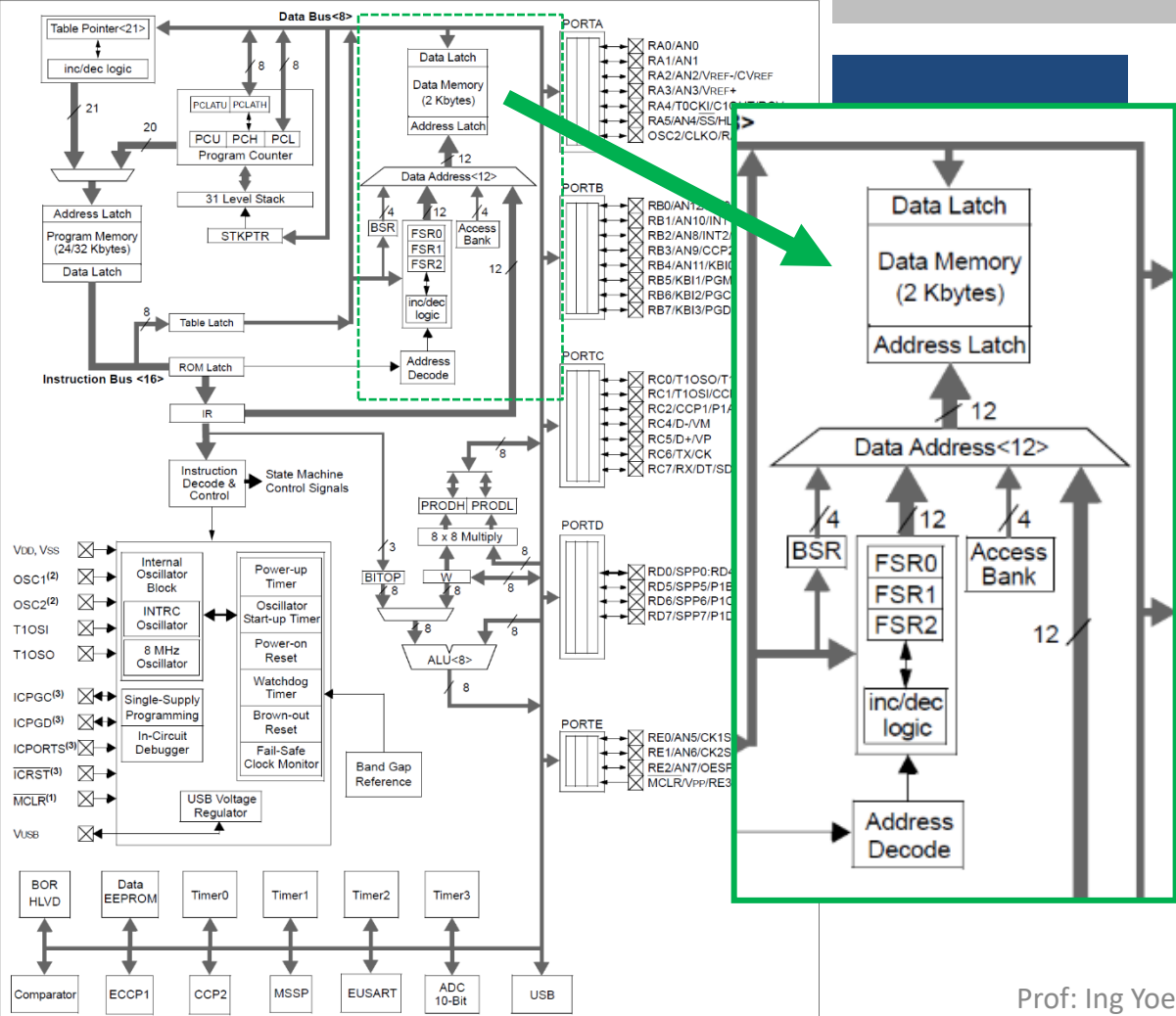




Arquitectura del PIC18F4

Diagrama de bloque

FIGURE 1-2: PIC18F4455/4550 (40/44-PIN) BLOCK DIAGRAM



Tema 3.1 Lenguaje de programación



Lenguaje de programación

Lenguaje de alto nivel

Lenguaje de bajo nivel

Lenguaje máquina

Lenguaje ensamblador

Notación de números en ensamblador



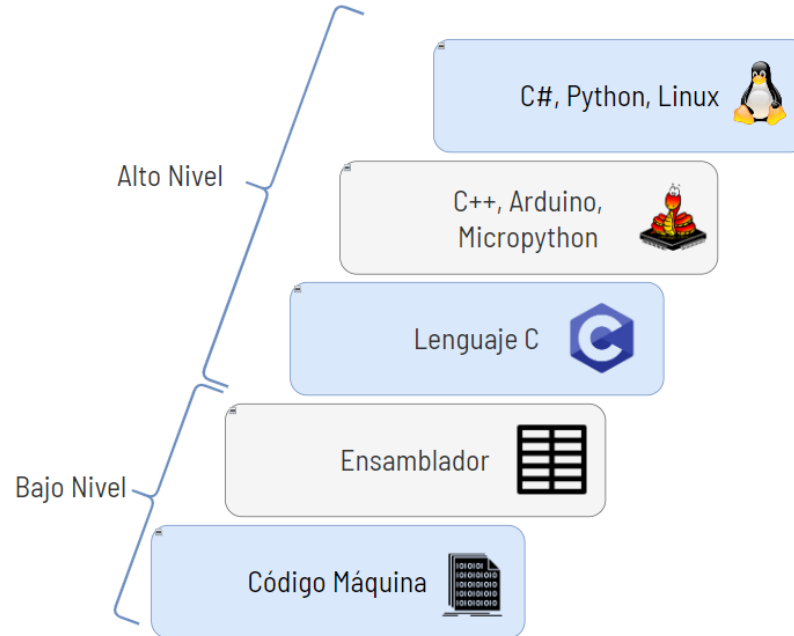
Lenguaje de programación

Lenguaje de programación

Es el conjunto de instrucciones que se debe escribir de manera lógica para realizar una tarea o acción.

Tipos :

- Lenguaje de alto nivel
- Lenguaje de bajo nivel





Lenguaje de programación

Lenguaje de alto nivel

- Se tratan de **lenguajes independientes de la arquitectura** del ordenador. Por lo que, en principio, un programa escrito en un lenguaje de alto nivel, se puede migrar de una máquina a otra sin ningún tipo de problema.
- Estos lenguajes **permiten al programador olvidarse por completo del funcionamiento interno de la(s) maquina(s) para la que está diseñando** el programa. Tan solo necesitan un traductor que entiendan el código fuente como las características de la máquina.



Lenguaje de programación

Lenguaje de bajo nivel

- Son lenguajes totalmente dependientes de la máquina, es decir que el programa que se realiza con este tipo de lenguajes no se pueden migrar o utilizar en otras máquinas.
- **Al estar prácticamente diseñados a medida del hardware, aprovechan al máximo las características del mismo.**
- Este lenguaje es mucho más rápido que los lenguajes de alto nivel.
- **Dentro de este grupo se encuentran:**
 - El lenguaje máquina
 - El lenguaje ensamblador



Lenguaje de programación

Lenguaje máquina

- Consiste en la combinación de 0 y 1 (cero y uno) para formar las ordenes entendibles por el hardware de la maquina.
- También conocido como «código maquina»
- **El único lenguaje que entienden los microcontroladores es el código máquina formado por ceros y unos del sistema binario.**
- son bastantes difíciles de manejar y usar, además de tener códigos fuente enormes donde encontrar un fallo es casi imposible.



Lenguaje de programación

Lenguaje ensamblador

- Es un derivado del lenguaje máquina.
- **Esta formado por abreviaturas de letras y números llamadas mnemónicos.**
- Se utiliza un programa traductor para poder pasar los programas escritos en lenguaje ensamblador a lenguaje máquina.



Lenguaje de programación

Nemónico

Es una abreviación de una palabra. Por ejemplo:

- **mov** es una abreviación de *move* (del ingles «move» que significa mover)
- **clr** es una abreviación de *clear* (del ingles «clear» que significa limpiar o borrar)

Las instrucciones están basadas en nemónicos pero de una forma mas abreviadas. Por ejemplo

movf es la abreviación de **mov** y **f** donde **f** es de la palabra *file* (del ingles «file» que significa registro) otro ejemplo es **clrf** que es la abreviación de **clear** y **file**

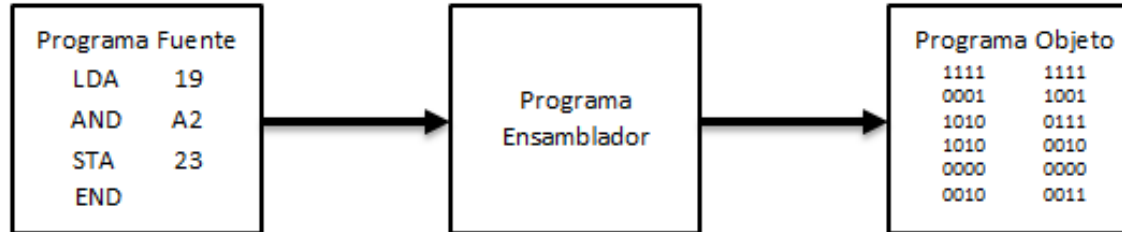
La mayoría de las inducciones están basadas en abreviaciones (nemónicos) a excepción de las instrucción **goto** y **call** (del ingles «goto» que significa ir para, y del ingles «call» que significa llamar) en este caso no es necesario abreviar la palabra por ser corta.



Lenguaje de programación

Instrucciones en lenguaje ensamblador

Están representados por nemónicos que luego son convertidos por el compilador a código máquina para poder ser ejecutado por el CPU del microcontrolador.



Tema 3.2 Instrucciones en Lenguaje Ensamblador del microcontrolador PIC18F4550



Contenido:

Instrucciones en lenguaje ensamblador

Set de instrucciones

Instrucciones que manejan registros

Instrucciones que manejan bits

Instrucciones de brincos o saltos

Instrucciones que manejan operados inmediatos

Instrucciones de control y Especiales

Sintaxis de las instrucciones



Instrucciones en Lenguaje Ensamblador

Notación de números en ensamblador

En el desarrollo de un programa en lenguaje ensamblador, se tienen que indicar valores numéricos. Estos deben estar en un formato claro para que el compilador interprete los datos.

Los formatos que se utilizan son:

- Binario
- Octal
- Decimal
- Hexadecimal
- ASCII

El menos utilizado es el octal. En el cuadro siguiente se muestra un ejemplo de cada una de las notaciones



Instrucciones en Lenguaje Ensamblador

Ejemplo de notación numérica en ensamblador

Notación	Ejemplo 1	Ejemplo 2	Ejemplo 3
Binario	B'11110010'	b'01010000'	
Decimal	D'41'	d'156 '	.25
Octal	O'32'	o'77'	
Hexadecimal	15H	1Fh	0x1F
ASCII	'A'	'b'	'#'



Instrucciones en Lenguaje Ensamblador

Simbología

Se presenta una simbología utilizada en la sintaxis de las instrucciones de los PIC de familia Mid-Range

SFR	Registro de Funciones Especificas
GPR	Registros de propósito General
f	Cualquier registro (SFR o GPR) de la RAM del PIC
w	Work register o Registro de trabajo
a	RAM access bit a = 0: Se utiliza el access bank (BSR es ignorado) a = 1: El banco se especifica en BSR
b	Posición de bit <0,7>
k	Campo literal, dato constante o etiqueta
d	Selección de destino; d = 0: el resultado se almacena en W d = 1: el resultado se almacenar en el registro F
PC	Program Counter o Contador de Programa
< >	Posición de bit dentro de un registro
Label	Nombre de una etiqueta



Instrucciones en Lenguaje Ensamblador

f

Como este microcontrolador maneja registro de 8 bit, entonces se tiene que

$$2^8 = 256$$

Eso significa que un registro puede tomar un valor entre 0 hasta 255

$$0 \leq f \leq 255$$

IMPORTANTE: Jamás se olviden que un registro de 8 bit esta comprendido en ese rango.



Instrucciones en Lenguaje Ensamblador

W

El registro de trabajo **W** es un registro de 8 bit que puede funcionar como un acumulador.

Este registro tiene una ubicación propia y es utilizado como registro auxiliar que puede servir como destino de el resultado de una operación. Por lo cual siempre esta presente en casi todas las instrucciones.

También se utiliza como uno de los operando en las instrucciones de operaciones lógicas o aritméticas



Instrucciones en Lenguaje Ensamblador

a

Se utiliza para especificar si la operación se realiza en el Access Bank o en el banco de memoria seleccionado mediante el registro BSR (Bank Select Register) .

Si el parámetro 'a' está configurado como 0 , la operación se realiza en el Access Bank , que incluye las primeras 128 direcciones del Banco 0 y las últimas 128 direcciones del Banco 15. Esto permite un acceso rápido a ciertas ubicaciones de memoria sin necesidad de cambiar el valor del registro BSR.

Si el parámetro 'a' está configurado como 1 , la operación se realiza en el banco de memoria seleccionado mediante el registro BSR . Esto es útil para acceder a ubicaciones de memoria fuera del Access Bank.



Instrucciones en Lenguaje Ensamblador

b

Se utiliza la letra **b** para indicar una posición de un bit dentro de un registro. como los registros son de 8 bit, entonces

$$0 \leq b \leq 7$$

Hay instrucciones que trabajan sobre un solo bit en específico, por lo cual hay que indicarlo en el programa. Mas adelante cuando se presenten las sintaxis de las instrucciones; cuando vean la letra b significa que se debe indicar el bit.



Instrucciones en Lenguaje Ensamblador

k

La letra k hace referencia a un dato de 8 bit o a una dirección de memoria de programa

Para facilitar las cosas, en vez de escribir la dirección, se utiliza una etiqueta la cual representa la posición de memoria de programa que queremos acceder.

Una Etiqueta es un nombre asignado por el programador (quien escribe el código). Puede ser en letras mayúsculas o minúsculas. Las etiqueta se utilizan principalmente en las instrucciones call y goto

Se utiliza la letra k en las sintaxis de las instrucciones ya sea para referirnos a una etiqueta o para un dato de 8bit según sea la instrucción.



Instrucciones en Lenguaje Ensamblador

d

El destino se representa con la letra d, existe dos posibles destino para el resultado de la operación; ya sea el propio registro (f) o el registro de trabajo w. Se puede indicar el destino de dos formas:

- Utilizando la letra f o el numero 1 para indicar que es el propio registro
- Utilizando la letra w o el numero 0 para indicar que es el registro de trabajo w

En caso de omitir el destino el compilador utiliza el destino de f por defecto.

Ejemplo: supongamos que queremos hacer la operación $A + W$ donde A es un registro y W es el registro de trabajo. Sea $A=3$ y $w=4$ entonces $A+w=7$

- si el destino es w ($d=w$) entonces después de la operación, w ya no valdrá 4 sino que tomara el valor del resultado de la operación, que en este caso seria $w=7$ y el registro A quedaría igual $A=3$
- si el destino es f ($d=f$) entonces después de la operación, A ya no valdrá 3 sino que tomara el valor del resultado de la operación, que en este caso seria $A=7$ y el registro w quedaría igual $w=4$



Instrucciones en Lenguaje Ensamblador

Set de instrucciones del PIC18F4550

Las instrucciones están divididas en 5 grupos:

1. Instrucciones que manejan registros
2. Instrucciones que manejan bit
3. Instrucciones que manejan operados inmediatos (literales «k»)
4. Instrucciones de control y Especiales

26.0 INSTRUCTION SET SUMMARY

PIC18F2455/2550/4455/4550 devices incorporate the standard set of 75 PIC18 core instructions, as well as an extended set of eight new instructions for the optimization of code that is recursive or that utilizes a software stack. The extended set is discussed later in this section.

26.1 Standard Instruction Set

The standard PIC18 instruction set adds many enhancements to the previous PIC MCU instruction sets, while maintaining an easy migration from these PIC MCU instruction sets. Most instructions are a single program memory word (16 bits) but there are four instructions that require two program memory locations.

Each single-word instruction is a 16-bit word divided into an opcode, which specifies the instruction type and one or more operands, which further specify the operation of the instruction.

The instruction set is highly orthogonal and is grouped into four basic categories:

- **Byte-oriented** operations
- **Bit-oriented** operations
- **Literal** operations
- **Control** operations



Instrucciones en Lenguaje Ensamblador

Instrucciones que manejan registros

Nemónico	Parámetro	Operación
ADDWF	f,d,a	Suma de f + w
ANDWF	f,d,a	f and w
CLRF	f,a,a	limpia f
COMF	f,d,a	complemento de f
DECF	f,d,a	decremento de f
INCF	f,d,a	Incremento de f
IORWF	f,d,a	f or w
MOVF	f,d,a	mover f a (d)
MOVWF	f,a	mover w a f
MOVFF	fs,fd	mover un registro a otro
MULWF	f,a	multiplica f con w el resultado en PRODH:PRODL
NOP	-	no operación
RLF	f,d,a	rotar f a la izquierda
RRF	f,d,a	rotar f a la derecha
SUBWF	f,d,a	resta de f – w
SWAPWF	f,d,a	Intercambiar nibble
XORWF	f,d,a	w xor w
DECFSZ	f,d,a	decremento de f y salta si es 0
INCFSZ	f,d,a	Incremento de f y salta si es 0



Instrucciones en Lenguaje Ensamblador

Instrucciones que manejan bits

Nemónico	Parámetro	Operación
BCF	f,b,a	Poner a 0 el bit
BSF	f,b,a	poner a 1 el bit
BTG	f,b,a	Bascula el bit
BTFSC	f,b,a	Testear el bit y salta si es 0
BTFSS	f,b,a	Testear el bit y salta si es 1



Instrucciones en Lenguaje Ensamblador

Instrucciones que manejan operados inmediatos

Nemónico	Parámetro	Operación
ADDLW	k	Suma de k+w
ANDLW	k	k and w
IORLW	k	k or w
MOVLW	k	Mover k hacia w
SUBLW	k	Resta de k - w
XORLW	k	k xor w



Instrucciones en Lenguaje Ensamblador

Instrucciones de control y Especiales

Nemónico	Parámetro	Operación
CALL	Label	Lllamar a sub-rutina
CLRWDI		refrescar al wdg
GOTO	Label	ir a sub-rutina
RETFIE		Retorno de interrupción
RETLW	k	Retorno de sub-rutina con w=k
RETURN		Retorno de sub-rutina
SLEEP		Modo CPU detenido



Instrucciones en Lenguaje Ensamblador

Para empezar a programar basta con aprenderse el siguiente grupo de instrucciones:

- 1.movlw
- 2.movwf
- 3.bcf
- 4.bsf
- 5.addwf
- 6.subwf
- 7.decfisz
- 8.btfsc
- 9.btfss
- 10.goto
- 11.call
- 12.return



Instrucciones en Lenguaje Ensamblador

Sintaxis de las instrucciones

- **Sintaxis:** explica cuales son los operando que requiere la instrucción (f=registro, d=destino, b=bit)
- **Operación:** es la acción que realiza el microcontrolador de forma automática y transparente al programador al ejecutarse la instrucción.
- **Ciclos:** es el tiempo que requiere el microcontrolador para ejecutar la instrucción. todas se ejecutan en un ciclo, excepto las de salto que requieren dos o tres ciclos.
- **Banderas:** es una señalización de algún evento interno del micro, por ejemplo las banderas **C,DC,Z** del registro STATUS.

Tema 3.3 Programa en lenguaje ensamblador



Contenido:

- ✓ Código fuente
- ✓ compilador
- ✓ Entorno de desarrollo integrado (IDE)
- ✓ MPLAB IDE
- ✓ Directivas



Programa en lenguaje ensamblador

código fuente

El código fuente es aquel que esta escrito por el programador, el cual contiene todas las instrucciones del programa, también suele contener comentarios que ayudan al programador a recordar lo que escribió. Esta compuesto por una sucesión de líneas de texto, se recomienda estructurar cada línea en cuatro columnas o campos:

- **Etiqueta:** Es una expresión alfanumérica escogidas por el programador para identificar una determinada línea de código, es decir, las etiquetas representan el valor de la posición de memoria en la línea que acompaña. Si una etiqueta es declarada, entonces cada vez que se haga referencia a ella se debe escribir igual, sino se producirá un error. Se puede escribir, ya sea en mayúscula o minúscula.
- **Instrucción:** Corresponde al nemónico que representa la instrucción junto con el operando.
- **Operando:** Contiene los datos, literal, etiqueta, dirección que requiera el nemónico o directiva.
- **Directiva:** Son indicaciones para el programa compilador.
- **Comentario:** Se considera comentario lo que este después de un punto y coma (;) y será ignorado por el programa compilador.
- **Macro:** Es un conjunto de instrucciones definidas por el programador.



Programa en lenguaje ensamblador

Ejemplo de código fuente

Columna_1 **Columna_2** **Columna_3** **Columna_4**
Etiquetas *memónico* *Operando* *;comentario*
ó Directivas

Ejemplo:

```

MOV LW    .128
MOVWF     NUM1      ;ASIGNAR VALOR DE 128 A NUM1
MOV LW    .16
SUBWF     NUM1,0    ;RESTAR 128-16 (NUM1-W) Y GUARDAR EN W
MOVWF     PORTD     ;MOVER EL VALOR DE W AL PUERTO D
BUCLE
GOTO      BUCLE     ;INICIAR BUCLE
END
  
```

*Nota: se recomienda usar la tecla de tabular para separar cada columna (TAB "la tecla que tiene dos flechas apuntando a ambos lados" que se encuentra alado de la tecla Q)



Programa en lenguaje ensamblador

Compilador

Es un programa de computadora que realiza varias acciones al código fuente de manera lógica y consecutiva; entre esas acciones están la de verificar la sintaxis del código. Además el compilador traduce el código escrito en lenguaje ensamblador a código máquina. Esta "traducción" se realiza en varias etapas de manera transparente al programador.

- **Error de sintaxis:** Es un error de escritura. El compilador puede indicar el error.
- **Error de lógica:** Es un error en la estructura del programa y ocurrirá durante la ejecución.



Programa en lenguaje ensamblador

MPLAB IDE

- Es un programa de computadora proporcionado por microchip para el desarrollo de proyecto con sus microcontroladores PIC. Se trata de un entorno de desarrollo integrado (IDE) con todas las herramientas para programar.
- Se encuentra gratuitamente disponible en la pagina de internet de microchip.



Programa en lenguaje ensamblador

Directivas en MPLAB IDE

Son comandos para el programa compilador. Se colocan en el código fuente, pero no son trasladados al programa que ejecutara el microcontrolador.

- LIST
- INCLUDE
- CONFIG
- CBLOCK

- ENDC
- ORG
- #DEFINE
- DT
- END